NeuralSpeech

Reconnaissance vocale embarquée sur Arduino Due

Soutenance 1 — ING3 S6 Calcul Embarqué — ECE Paris

Gaspard BOUCHARLAT
Albin PACAUT
Maxence MICHEL
Timothy DERNBACH

Date — 2026

Le défi : reconnaître la voix sans GPU

L’objectif est de classer un mot prononcé par **une IA tournant intégralement sur un microcontrôleur 84 MHz**, sans aucun framework lourd.

Plateforme imposée

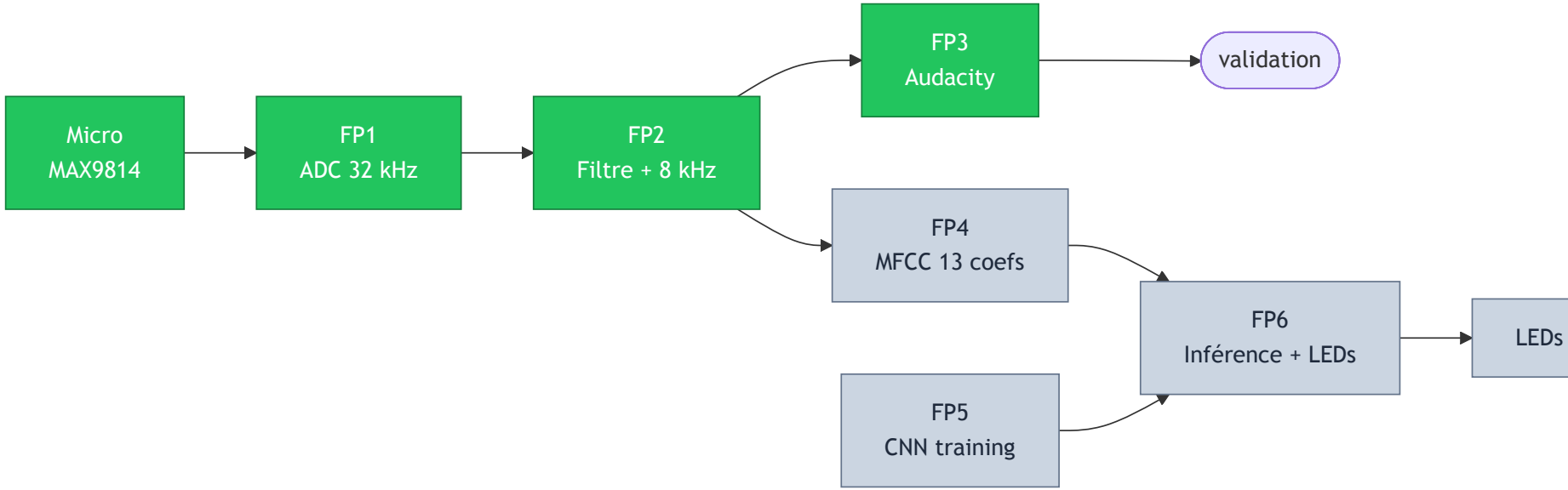
- Arduino **Due** (SAM3X8E)
- Cortex-M3 — **pas de FPU**
- 96 Kio SRAM, 512 Kio Flash

Cahier des charges

- 9 exigences techniques (ET1–ET9)
- Pipeline complet : ADC → MFCC → CNN → LEDs
- Malus −12 si non Arduino Due

ET1–ET9 : exigences techniques du sujet NeuralSpeech 2026 V1.3

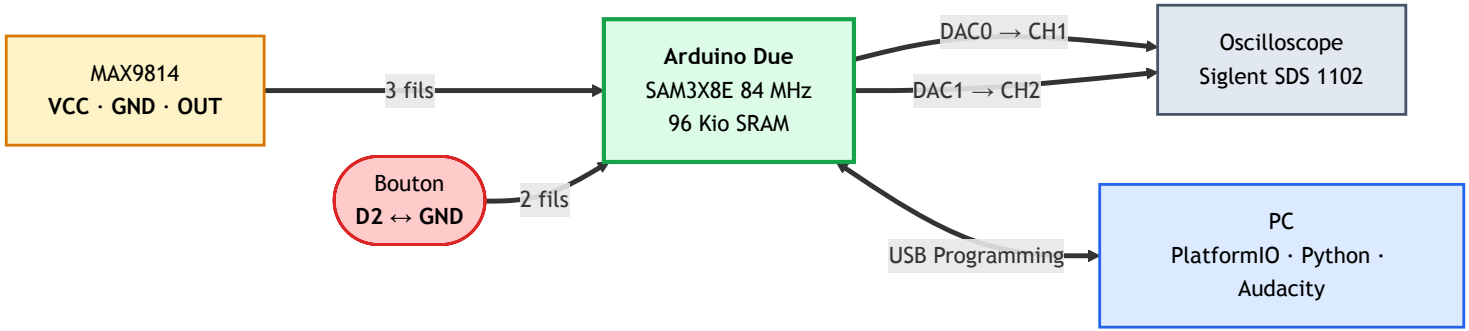
Pipeline NeuralSpeech : 6 fonctions, 3 closes



Soutenance 1 (aujourd'hui) — ✓ FP1, FP2, FP3

Soutenance 2 — FP4, FP5, FP6

Architecture matérielle : 4 fils + 1 bouton + 1 micro



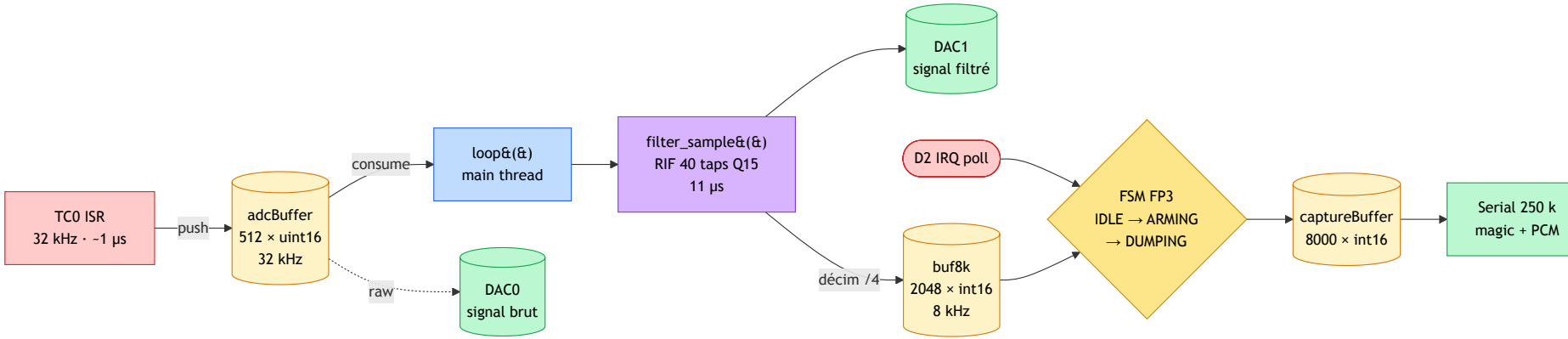
Pinout retenu (Arduino Due)

Broche	Fonction
A0	ADC entrée audio (MAX9814 OUT)
D2	Bouton FP3 (pull-up interne)
DAC0 (pin 66)	Signal **brut** → CH1 oscillo

Choix d'intégration

- MAX9814 alimenté en **3,3 V** (Due *non* 5 V tolérante)
- Bouton **sans résistance externe** (`INPUT_PULLUP` interne)
- Aucun shield → câblage direct breadboard
- **Masse commune** Due ↔ GBF ↔ oscilloscope

Architecture logicielle : 3 buffers circulaires, 2 contextes



Pourquoi 3 buffers circulaires ?

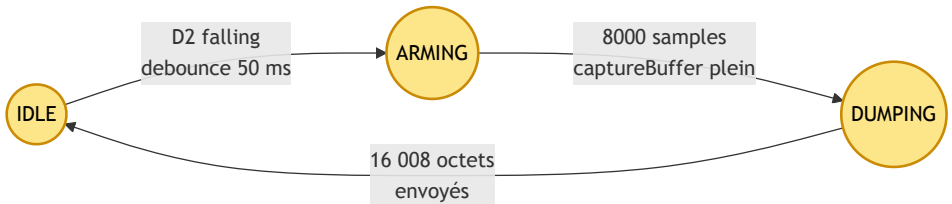
Chacun découple un couple producteur/consommateur :

1. ISR → loop (audio brut)
2. loop → loop (FIR delay line)
3. loop → FSM (audio filtré 8 kHz)

Pourquoi filtrer dans `loop()` et non l'ISR ?

- ISR doit rester **< 31,25 µs** (ET3)
- Filtre 40 taps Q15 ≈ 11 µs → tient en ISR mais alourdit la latence
- Loop a 32 cycles ISR de marge → prélève à son rythme
- Découplage strict : ISR = capture, loop = traitement

Algorigramme : ISR TC0 + state machine FP3



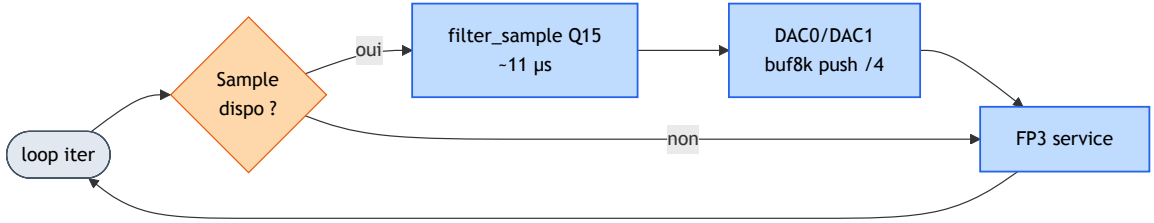
ISR TC0 — producteur 32 kHz

ISR ultra-courte (~1 µs) : pas de calcul, juste lecture ADC + push buffer. Garantit Fe stable à 0,04 % d'erreur près.

FSM FP3 — bouton D2

Non bloquante : la chaîne FP1+FP2 continue de filtrer pendant ARMING (3 s de capture) et DUMPING (~0,64 s de transfert série).

Algorigramme : `loop()` — flux par échantillon



3 étapes par échantillon (mode IDLE)

1. **Sample dispo ?** — pop ADC buffer si oui
2. **Filtrage Q15** — 40 MAC en ~11 µs, sortie sur DAC0 (brut) et DAC1 (filtré), 1/4 → buf8k
3. **FP3 service** — polling bouton D2 + envoi série (1 octet/tour)

Coût total < 15 µs

→ Largement sous le budget ET3 (31,25 µs / sample).

→ La loop tourne plus vite que l'ISR ne produit.

→ `buf_used = 0-1 / 512` en régime permanent : aucune perte d'échantillon.

FP1 — Échantillonnage à Fe = 32 kHz exacts

Choix : Timer Counter (TC0) + ADC plutôt qu'ADC free-running.

- Cadence isochrone, indépendante du CPU
- Conforme à l'exigence "interruption sur timer"

Calcul du registre RC

$$\mathrm{RC} = \left\lfloor \frac{F_{MCK/8}}{F_e} \right\rfloor = \left\lfloor \frac{10\,500\,000}{32\,000} \right\rfloor = 328$$

$$F_e^{\text{réelle}} = \frac{10\,500\,000}{328} = 32\,012\ \mathrm{Hz}$$

Erreur : **0,04 %**.

```c
void TC0_Handler(void) {
  // 1) Acquittement IRQ (lecture du SR)
  TC0->TC_CHANNEL[0].TC_SR;

  // 2) Conversion ADC bloquante minimale
  ADC->ADC_CR = ADC_CR_START;
  while (!(ADC->ADC_ISR & (1u << 7))) {}
  uint16_t s = ADC->ADC_CDR[7] & 0x0FFF;

  // 3) Push buffer circulaire (1 cycle)
  adcBuffer[bufHead] = s;
  bufHead = (bufHead + 1) & BUFFER_MASK;
  sampleCount++;
}
```

ISR ultra-courte : **~ 1 µs**, déterministe.

ET1 validée : Fe = 32 012 Hz, stable

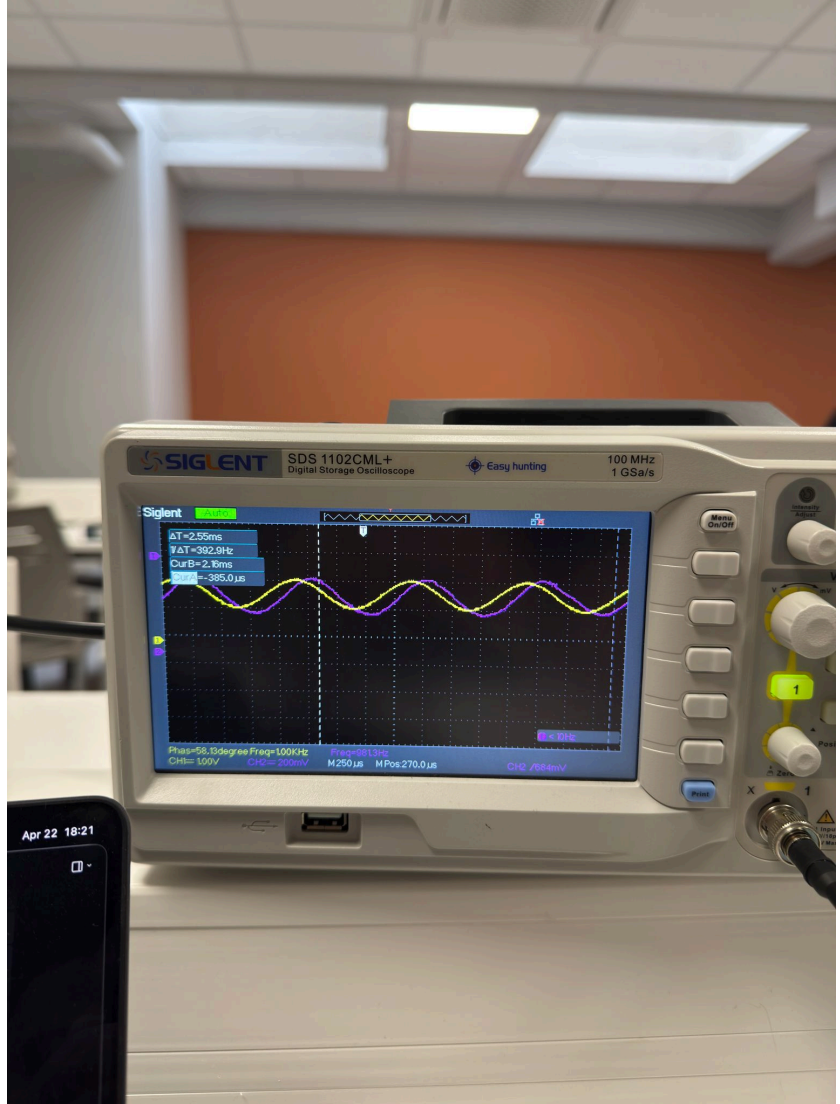
3 preuves complémentaires :

1. **Console série** — comptage des ISR sur 1 seconde
   → Te = 1 / 32 012 = **31,238 µs**

2. **Reconstruction oscillo** — sinus 1 kHz
   CH1 (entrée) ≡ CH2 (DAC0) — *voir image*

3. **Buffer non saturé** — `buf_used = 0-1 / 512`
   la loop consomme plus vite que l'ISR ne produit

Erreur de fréquence : **0,04 %** vs cible 32 kHz.



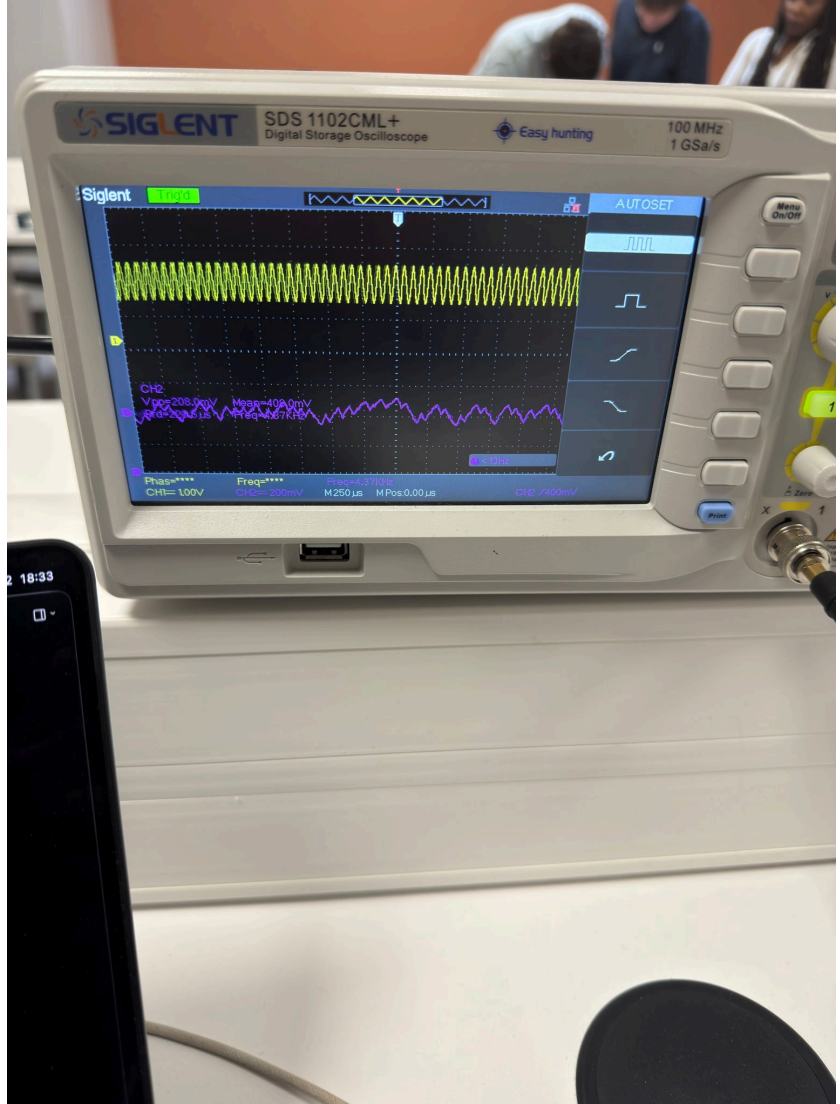
Démo Nyquist : à $F_e/2$ la reconstruction casse

Test pédagogique : injection à **16 kHz** = limite théorique de Nyquist.

- CH1 (entrée brute) → toujours bien présent
- CH2 (reconstruction DAC) → **plus reconnaissable**, amplitude divisée par 5

À la limite, 2 échantillons par période **ne suffisent plus** à capturer la phase du signal.

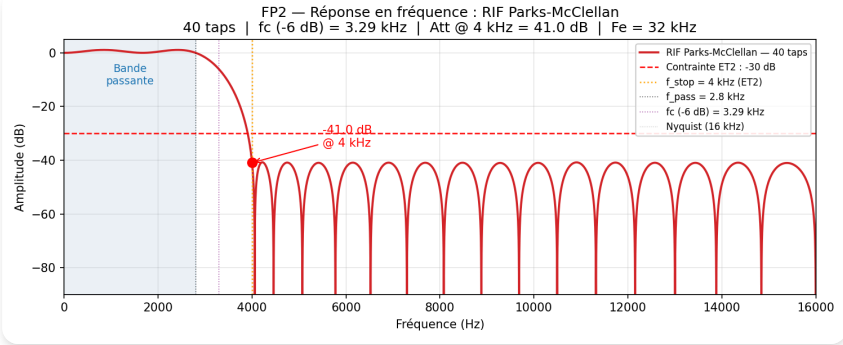
Conclusion : tout signal au-dessus de 4 kHz dans la bande utile (après décimation /4) **se replie** dans la voix.



FP2 — RIF Parks-McClellan, 40 taps optimaux

Méthode retenue — Parks-McClellan equiripple (T-Filter)

Optimal au sens « plus petit ordre pour un gabarit donné » → **40 taps** (vs 97 avec méthode des fenêtres).

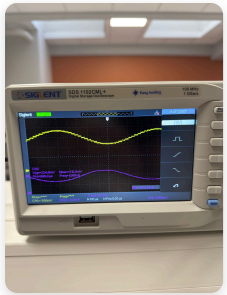


Réponse en fréquence du RIF 40 taps.
Le seuil ET2 (−30 dB, ligne rouge) est franchi avec ≥ 11 dB de marge.

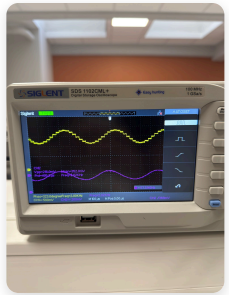
Gabarit visé

Bande	Plage	Critère
Passante	0 – 2,8 kHz	gain 1, ripple 1 dB

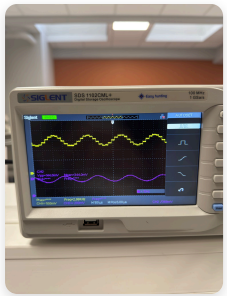
ET2 démontrée à l'oscilloscope (DAC0 vs DAC1)



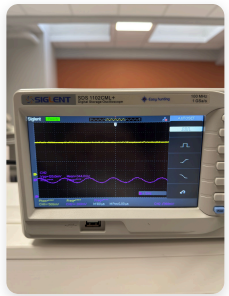
1 kHz — CH1 = CH2 (passe)



2 kHz — toujours en bande passante



3 kHz — entrée en transition



4 kHz — CH2 effondré → ET2 ✓

DAC0 = signal brut (CH1) · DAC1 = signal filtré (CH2) · GBF 1 V_{pp} sinus

ET3 : 900 µs → 11 µs en 4 fixes

Budget ET3 : < 31,25 µs par échantillon

#	Fix	Effet
1	Q15 fixed-point	float émulé éliminé
2	`build_unflags=-Os`	`-O2` enfin actif
3	Mesure DWT cycle counter	sonde fiable
4	ADC prescaler 41 → 1	ISR 20 µs → 1 µs

```
=== Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
19:10:39.136 > === NeuralSpeech FP1+FP2 — ADC 32 kHz + filtre RIF + buf 8 kHz ===
19:10:39.140 > FE             : 32000 Hz
19:10:39.140 > FE_OUT         : 8000 Hz
19:10:39.140 > FILTER_TAPS    : 40
19:10:39.140 > SUBSAMPLE      : 1/4
19:10:39.144 > BUF ADC        : 512 samples
19:10:39.144 > BUF 8kHz       : 2048 samples (int16)
19:10:39.144 > FIR buf size   : 128 (puissance de 2)
19:10:39.148 > Configuration ADC, DAC, TC...
19:10:39.148 > Demarrage acquisition + filtrage...
19:10:40.144 > [FP1] Fe_reelle=32011 Hz | samples=32012 | buf_used=0/512
19:10:40.144 > [FP2] filter_us_avg=11.29 max=14.64 | buf8k_used=1859/2048 | taps=40
19:10:41.143 > [FP1] Fe_reelle=32011 Hz | samples=32012 | buf_used=0/512
19:10:41.147 > [FP2] filter_us_avg=11.29 max=14.57 | buf8k_used=1670/2048 | taps=40
19:10:42.142 > [FP1] Fe_reelle=32012 Hz | samples=32013 | buf_used=0/512
19:10:42.147 > [FP2] filter_us_avg=11.29 max=14.45 | buf8k_used=1481/2048 | taps=40
19:10:43.142 > [FP1] Fe_reelle=32011 Hz | samples=32012 | buf_used=1/512
19:10:43.146 > [FP2] filter_us_avg=11.29 max=14.34 | buf8k_used=1292/2048 | taps=40
19:10:44.145 > [FP1] Fe_reelle=32012 Hz | samples=32013 | buf_used=0/512
19:10:44.146 > [FP2] filter_us_avg=11.29 max=14.36 | buf8k_used=1103/2048 | taps=40
19:10:45.145 > [FP1] Fe_reelle=32011 Hz | samples=32012 | buf_used=1/512
19:10:45.145 > [FP2] filter_us_avg=11.29 max=14.63 | buf8k_used=914/2048 | taps=40
19:10:46.145 > [FP1] Fe_reelle=32012 Hz | samples=32013 | buf_used=0/512
19:10:46.145 > [FP2] filter_us_avg=11.29 max=14.28 | buf8k_used=725/2048 | taps=40
19:10:47.144 > [FP1] Fe_reelle=32011 Hz | samples=32012 | buf_used=0/512
19:10:47.148 > [FP2] filter_us_avg=11.29 max=14.57 | buf8k_used=536/2048 | taps=40
19:10:48.143 > [FP1] Fe_reelle=32012 Hz | samples=32013 | buf_used=0/512
19:10:48.147 > [FP2] filter_us_avg=11.29 max=14.27 | buf8k_used=348/2048 | taps=40
19:10:49.143 > [FP1] Fe_reelle=32011 Hz | samples=32012 | buf_used=0/512
19:10:49.147 > [FP2] filter_us_avg=11.29 max=14.58 | buf8k_used=159/2048 | taps=40
19:10:50.142 > [FP1] Fe_reelle=32012 Hz | samples=32013 | buf_used=0/512
19:10:50.146 > [FP2] filter_us_avg=11.29 max=14.47 | buf8k_used=2018/2048 | taps=40
19:10:51.146 > [FP1] Fe_reelle=32011 Hz | samples=32012 | buf_used=1/512
19:10:51.146 > [FP2] filter_us_avg=11.29 max=14.30 | buf8k_used=1829/2048 | taps=40
19:10:52.145 > [FP1] Fe_reelle=32012 Hz | samples=32013 | buf_used=0/512
19:10:52.146 > [FP2] filter_us_avg=11.29 max=14.57 | buf8k_used=1640/2048 | taps=40
19:10:53.145 > [FP1] Fe_reelle=32012 Hz | samples=32013 | buf_used=0/512
19:10:53.145 > [FP2] filter_us_avg=11.29 max=14.59 | buf8k_used=1451/2048 | taps=40
19:10:54.144 > [FP1] Fe_reelle=32011 Hz | samples=32012 | buf_used=0/512
19:10:54.148 > [FP2] filter_us_avg=11.29 max=14.59 | buf8k_used=1262/2048 | taps=40
19:10:55.143 > [FP1] Fe_reelle=32012 Hz | samples=32013 | buf_used=0/512
19:10:55.148 > [FP2] filter_us_avg=11.29 max=14.32 | buf8k_used=1074/2048 | taps=40
19:10:56.143 > [FP1] Fe_reelle=32011 Hz | samples=32012 | buf_used=1/512
19:10:56.147 > [FP2] filter_us_avg=11.29 max=14.50 | buf8k_used=885/2048 | taps=40
19:10:57.142 > [FP1] Fe_reelle=32012 Hz | samples=32013 | buf_used=0/512
```

Console série mesurée par le compteur de cycles matériel DWT (1 cycle de lecture, zéro overhead).

> 💡 **Leçon DSP embarqué** : sur Cortex-M3 sans FPU, tout traitement temps-réel passe par du fixed-point.

FP3 — Bouton, capture 1 s, WAV vers Audacity

Protocole binaire

Magic header `0xAA55AA55` + uint32 length + 16 000 octets PCM int16 + footer `0xDEADBEEF`.

Logs ASCII suspendus pendant DUMPING pour ne pas polluer le flux.

ET4 — preuve auditive

À l'écoute du `.wav` dans Audacity :

- ✓ on **reconnaît** « Électronique »
- ✓ 4 syllabes distinctes
- ✓ bande utile 0–4 kHz préservée
- ✓ pas de saturation, pas de clic

Bilan Soutenance 1 — 4 ET sur 9 validées

ET validées

ET	Critère	Mesuré
ET1	Fe = 32 kHz timer	**32 012 Hz**
ET2	Att. ≥ 30 dB @ 4 kHz	**−41 dB** théorique
ET3	Filtrage < 31 µs	**11,3 µs avg**
ET4	« Électronique » Audacity	✓ audible

Ressources Arduino Due

Ressource	Utilisé	Capacité	%
SRAM	24 Kio	96 Kio	**24,5 %**
Flash	27 Kio	512 Kio	**5,2 %**
ISR TC0	1 µs	31,25 µs	3 %

→ **75 Kio SRAM libre** pour FP4 (MFCC) + FP6 (inférence CNN).

Roadmap Soutenance 2 : FP4 MFCC (62×13 par seconde) → FP5 CNN training Python → FP6 inférence embarquée + LEDs → < 5 % d'erreur sur 10 mots.

Questions ?

Démo disponible — câble micro / bouton / oscilloscope sur le bureau.

Gaspard BOUCHARLAT
Albin PACAUT
Maxence MICHEL
Timothy DERNBACH

Code source : github.com/gascar9/NeuralSpeech

Backup — Filtre RIF Q15

```c
int16_t filter_sample(uint16_t new_sample) {
  // 1) Centrer ADC autour de 0
  int16_t centered = (int16_t)new_sample - 2048;

  // 2) Pousser dans buffer circulaire RIF
  firBuf[firBufHead] = centered;
  firBufHead = (firBufHead + 1) & FIR_BUF_MASK;

  // 3) Convolution Q15 — 40 MAC int32
  int32_t acc = 0;
  uint32_t idx = (firBufHead - 1) & FIR_BUF_MASK;
  for (uint32_t k = 0; k < FILTER_TAPS; k++) {
    acc += (int32_t)FILTER_COEFS_Q15[k]
         * (int32_t)firBuf[idx];
    idx = (idx - 1) & FIR_BUF_MASK;
  }
  return (int16_t)(acc >> 15);  // Q15 → Q0
}
```

Backup — Mesure DWT cycle counter

```c
// Setup unique
SCB_DEMCR  |= DEMCR_TRCENA;
DWT_CYCCNT  = 0;
DWT_CTRL   |= DWT_CTRL_CYCCNTENA;

// Mesure : 1 cycle hardware par lecture
uint32_t c0 = DWT_CYCCNT;
int16_t y = filter_sample(x);
uint32_t c1 = DWT_CYCCNT;

uint32_t us_x100 = ((c1 - c0) * 100UL) / 84UL;
```

Pourquoi pas `micros()` **?** Sur Due, `micros()` exécute un retry-loop sensible aux ISR TC0. Coût mesuré : **20–30 µs par appel**, qui polluait notre mesure de filtrage initial à hauteur de 60 µs.

DWT = lecture directe du compteur de cycles ARM Cortex-M3.